**牛客周赛 Round 76 题解**

> https://ac.nowcoder.com/acm/contest/99990

# A.小红出题

> https://ac.nowcoder.com/acm/contest/99990/A

小红每周一到周五上班，工作日每天都要出 $3$ 道题目。
已知小红入职当天为星期一，并且已经在职 $n$ 天。
请问她总共出了几道题目?

**输入描述:**

第一行有一个整数 $n$ ( $1 \le n \le 10^3$ )

**输出描述:**

输出一个整数，代表题目总数 。

## 题解

对于完整的一周，小红要出 $5 \times 3 = 15$ 道题，剩余的天数中小红最多出 $5$ 天题目。

代码如下

```
n = int(input())
print(n//7*15 + min(n%7, 5)*3)
```

# B 串串香

> https://ac.nowcoder.com/acm/contest/99990/B

给定一个长度为 $n$ ，仅包含小写字母的字符串 $s$ 。

请你构造出一个非空字符串 $t$ ，使得它在 $s$ 中作为**子串**出现的次数最多。

子串是指，从原字符串中连续截取一些字符，得到的新字符串。

**输入描述:**

第一行有一个整数 $n$ ( $1 \le n \le 10^5$ ) 。

第二行有一个字符串 $s$ ，字符串仅包含小写字母。

**输出描述:**

输出一个字符串，代表构造得到的字符串 $t$ 。

如果有多个字符串符合条件，输出任意一个即可。

## 题解

不难想到，要找到出现次数最多的子串，子串长度为 $1$ 时一定能找到一个。

证明如下:

如果有字符串 $t1$ 满足 $|t| > 1$且 $t1$ 出现的次数是最多的， 那么 $t1$ 的子串 $t2$ 出现的次数 一定和 $t1$ 相同。

不断细分下去最终一定能得到一个长度为 1 的子串。

代码如下

```
from collections import Counter
n = int(input())
s = input()

mm = Counter(s)
for k in mm:
    if mm[k] == max(mm.values()):
        print(k)
        break
```

# C.小红的gcd

> https://ac.nowcoder.com/acm/contest/99990/C

小红有一个长度为 $n$ 的数组，她希望数组元素之和越少越好。

她可以进行**任意次**操作，每次选择数组中的两个元素 $a_i$ 和 $a_j$ ，令 $a_i = a_j = \gcd(a_i, a_j)$ 。

所有操作结束后，请你输出**最小**的数组元素之和。

**输入描述:**

第一行有一个整数 $n$ ( $1 \le n \le 10^5$ ) 。

第二行有 $n$ 个整数 $a_i$ ( $1 \le a_i \le 10^9$ ) 。

**输出描述:**

第一行有一个整数 $n$ ( $1 \le n \le 10^5$ ) 。

第二行有 $n$ 个整数 $a_i$ ( $1 \le a_i \le 10^9$ ) 。

## 题解

对于 $a, b, c$ 有 $gcd(a, b, c) = gcd(gcd(a, b), c)$

因为次数不限，我们总能把所有元素都变为他们的公共 $gcd$，和即为 $gcd * n$

代码如下

```
from math import gcd

n = int(input())
a = list(map(int, input().split()))
num = a[0]
for i in range(1, len(a)):
    num = gcd(num, a[i])
    if num == 1:
        break
print(num*n)
```

# D.奇偶调整

**本题是假题，略过**

UPD：**本题假了**，hack数据如下：

2 2 1

11 9

正确答案应该为 13 ， 而不是 14 。

# E.幂次进近

> https://ac.nowcoder.com/acm/contest/99990/E

给定 $t$ 次询问，每次询问给出两个正整数 $n$ 和 $k$ 。

请你找到最小的**正整数** $m$ ，使得 $n-m^k$ 的**绝对值**最小。

**输入描述:**

第一行有一个整数 $t$ ( $1 \le t \le 10^5$ ) 。

随后 $t$ 行，每行两个整数 $n, k$ ( $1 \le n, k \le 10^{18}$ ) 。

**输出描述:**

输出 $t$ 行，每行一个正整数 $m$ 。

## 题解

当 $k >= 61$ 时, $2^{61} = 2305843009213693952 > 2 * 10^{18}$, 此时答案一定为 $1$。

当 $k = 1$ 时，答案为 $n$

当 $1 < k < 61$时，答案为 $\lfloor n^{\frac{1}{k}} \rfloor^k$ 和 $(\lfloor n^{\frac{1}{k}} \rfloor + 1)^k$ 中离 $n$ 近的那一个。

但是$\lfloor n^{\frac{1}{k}} + 1 \rfloor^k$ 会超过 $64$ 位整数，我们可以把比较的式子做一定的变化

$$
\begin{aligned}
&(\lfloor n^{\frac{1}{k}} \rfloor + 1)^k - n >= n - \lfloor n^{\frac{1}{k}} \rfloor^k \\
&\Leftrightarrow (\lfloor n^{\frac{1}{k}} \rfloor + 1)^k - 2 \cdot n + \lfloor n^{\frac{1}{k}} \rfloor^k >= 0 \\
&\Leftrightarrow (1 + \frac{1}{\lfloor n^{\frac{1}{k}} \rfloor^k})^k - 2 \cdot \frac{1}{\lfloor n^{\frac{1}{k}} \rfloor^k} + 1 >= 1
\end{aligned}
$$

计算时使用 $long\ double$ 来确保精度不会出错

当然还有一种更简单的方法是用 $python$ （

代码如下

```
from math import floor, ceil
t = int(input())
for _ in range(t):
    n, k = map(int, input().split())
    if k >= 61:
        print(1)
    elif n==1 or k == 1:
        print(n)
    else:
```

```
        x = n ** (1.0 / k)
        m1 = floor(x)
        m2 = ceil(x)
        m1 = max(m1, 1)
        if n - m1 ** k <= m2**k - n:
            print(m1)
        else:
            print(m2)
```

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

inline ll read(){
    ll s = 0, w = 1; char ch = getchar();
    while (ch < 48 || ch > 57) { if (ch == '-') w = -1; ch = getchar(); }
    while (ch >= 48 && ch <= 57) s = (s << 1) + (s << 3) + (ch ^ 48), ch =
getchar();
    return s * w;
}

inline void pt(ll x){if(x<0) putchar('-'),x=-x;if(x>9)
pt(x/10);putchar(x%10+'0');}
void print(ll x){pt(x), puts("");}

ll qpow(ll x, ll p) {
        ll res = 1;
        for (; p; p >>= 1, x = x * x)
            if (p & 1) res = res * x;
        return res;
}

void solve(){
    ll n = read(), k = read();
    if (k>= 61){
        print(1);
    }
    else if (k > 1){
        ll m = static_cast<ll>(pow(n, (long double)1.0/k));
        ll mk = qpow(m, k);
        long double n1 = (long double)1.0 * n/mk;
        long double n2 = pow((1+(long double)1.0/m), k);
        if (m == 0){
            print(1);
        }
        else if (n2 - 2*n1 + 1<0){
            print(m+1);
        }
        else{
            print(m);
        }
    }
    else{
        print(n);
```

```
    }
}

int main(){
    ll t = read();
    while(t--){
        solve();
    }
}
```

# F.同位序列

定义 $f(x)$ 为 $x$ 在二进制表示下的 $1$ 的个数。

例如 $f(7)=3$，$f(8)=1$，$f(9)=2$。

定义 $g(x)$ 为 第一个比 $x$ 大的数字 $y$，使得 $f(x)=f(y)$。

例如 $g(1)=2$，$g(2)=4$，$g(3)=5$。

给你一个长度为 $n$ 的数组 $a$，第 $i$ 个元素为 $a_i$。

我们希望从数组 $a$ 中挑选出一些元素，构造一个长度为 $m$ 的同位序列 $b$。

同位序列 满足如下条件：

对于所有的 $j\in[2,m]$，都有 $b_j=g(b_{j-1})$。

当然，同位序列越长越好，请你最大化其长度 $m$，然后输出 $m$ 和对应的同位序列。

如果有多解，输出任意一个即可。

**输入描述:**

第一行有一个整数 $n$ ( $1\le n\le 10^5$ )。

第二行有 $n$ 个整数 $a_i$ ( $1\le a_i\le 10^9$ )。

**输出描述:**

第一行输出一个整数 $m$，代表 最长的同位序列 的长度。

第二行输出 $m$ 个整数，代表同位序列中的元素。

## 题解

本题的难点其实是如何$O(logn)$ 或 $O(1)$ 复杂度内求出 $g(x)$

本质上 $g(x)$ 其实是 在不改变 $1$ 和 $0$ 的数目下的下一个排列。

注意 $g(x)>x$

这里给出几种求法

```
def g(x):
   # O(logn)
    f = False
    for i in range(35):
        if x >> i & 1 == 0:
            if f:
```

```
                x |= 1 << i
                v = x & ((1 << i) - 1)
                x -= v
                x |= (1 << bin(v).count('1') - 1) - 1
                return x
        else:
            f = True

 def g2(x):
    # O(logn)
    c = x
    c0 = 0
    while (c & 1) == 0 and c != 0:
        c0 += 1
        c >>= 1
    c1 = 0
    while (c & 1) == 1:
        c1 += 1
        c >>= 1
    p = c0 + c1
    x |= 1 << p
    x &= ~((1 << p) - 1)
    x |= (1 << (c1 - 1)) - 1
    return x

def g3(x):
    # O(1)
    o=x&-x
    t=x+o
    c=t&-t
    m=((c//o)>>1)-1
    ans=t|m
    return ans
```

我们预处理出每一个 $g(a_i)$，在从大到小计算以$a_i$为开头的最长序列长度。

若$g(a_i)$ 在序列中，则$l(a_i)$为$l(g(a_i))+1$；否则为 $1$。

我们选取任意一个最长的序列，在还原他即可

代码如下

```
n = int(input())
nums = list(map(int, input().split()))

def g(x):# g(x)略去，详见上面的代码

nex = {x: g(x) for x in nums}

maxlen = {}
nums.sort(reverse=True)

for i in nums:
    maxlen[i] = 1 if nex[i] not in maxlen else maxlen[nex[i]] + 1 # 哈希表in是O(1)
```

```python
mx = max(maxlen.values())
for i in maxlen:
    if maxlen[i] == mx:
        ans = [i]
        for _ in range(mx - 1):
            ans.append(nex[ans[-1]])
        print(len(ans))
        print(*ans)
        break
```